

Unit 4.2: Numerical Methods

Dr. Manoj C. Patil

2026-09-19

Learning Objectives

By the end of this unit, a student should be able to

- explain why closed-form solutions to nonlinear equations and integrals are the exception rather than the rule in statistical practice, motivating numerical methods,
- implement and prove the linear convergence of the bisection method for locating a root,
- implement and prove the quadratic convergence of the Newton-Raphson method, and extend it to systems of nonlinear equations using the Jacobian matrix,
- derive the trapezoidal and Simpson's rule quadrature formulas, their error terms, and their convergence rates,
- implement all four methods in R and empirically verify their theoretical convergence rates,
- recognize the role of these numerical methods within statistical computing, particularly maximum likelihood estimation and numerical integration of intractable expectations.

Prerequisites

Students should be comfortable with

- single-variable and multivariable calculus, including Taylor's theorem with remainder and partial derivatives,
- basic linear algebra, including matrix inversion and determinants of 2×2 matrices,
- the concept of a maximum likelihood estimator and the score equation,
- R programming, including writing iterative loops and simple matrix operations.

Introduction

Much of theoretical statistics is stated in a form that presumes an equation can be solved, or an integral evaluated, exactly. The maximum likelihood estimator is defined as the value that sets the score function to zero; the sampling distribution of a statistic is often described through an integral of a density; a quantile is defined implicitly as the value x solving $F(x) = u$. In practice, closed-form solutions to these problems are the exception. The maximum likelihood equations for all but the simplest models are nonlinear and admit no algebraic solution; most integrals of interest in probability and statistics — even something as basic as the standard Normal cumulative distribution function — have no elementary antiderivative. Numerical methods supply the machinery to solve such problems to any desired accuracy using arithmetic operations alone.

This unit develops the two most fundamental families of numerical methods used throughout statistical computing. **Root-finding methods** — the bisection method and the Newton-Raphson method — locate the value or values of x satisfying $f(x) = 0$, and extend naturally to systems of nonlinear equations arising, for instance, from multiparameter score equations. **Numerical integration methods** — the trapezoidal rule and Simpson's rule — approximate a definite integral using only evaluations of the integrand at a finite set of points, and form the deterministic counterpart to the Monte Carlo integration methods developed in

Unit 4.3. The bisection method dates to the ancient method of exhaustion and was formalized in the 19th century; the Newton-Raphson method traces to Isaac Newton (1669) and Joseph Raphson (1690); Simpson's rule, though named for Thomas Simpson (1743), was known earlier to Johannes Kepler. Despite their age, these methods remain the computational engines inside `optim()`, `nlm()`, `uniroot()`, and `integrate()` in R, and inside virtually every statistical software package's maximum likelihood fitting routines.

The practical importance of this material is immediate and pervasive: fitting a generalized linear model, computing a profile likelihood confidence interval, or evaluating the normalizing constant of a posterior density all reduce, at some point, to root-finding or numerical integration. Understanding how these building blocks work, and their limitations, is essential to using statistical software correctly and to diagnosing convergence failures when they occur.

Intuition

Consider searching for a lost item along a long corridor, where a helper can only tell you, at any point you ask, whether the item is to your left or to your right. The most conservative strategy is to check the midpoint of the corridor, learn which half the item is in, and repeat on that half — halving the search region at every step. This is exactly the logic of the **bisection method**: given two points at which a continuous function f has opposite signs, the Intermediate Value Theorem guarantees a root lies between them, and evaluating the midpoint and keeping whichever half still brackets a sign change halves the uncertainty at every step, guaranteed, but never faster than that.

A sighted searcher who can also see how steeply the corridor's brightness is changing could do better: standing at a point and observing both the reading and its rate of change, they could extrapolate — assuming the trend continues in a straight line — to where the reading would hit zero, and jump there directly rather than merely halving the interval. This is the idea behind the **Newton-Raphson method**: at the current point x_n , approximate the function by its tangent line, and jump to where that tangent line crosses zero. When the function is well approximated by its tangent line near the root — which a smooth function always is, sufficiently close to a simple root — this extrapolation converges far faster than repeated halving.

A parallel intuition underlies numerical integration. The exact area under a curve is unavailable in closed form, but the area under a *simple* curve — a straight line, or a parabola — passing through a few known points of the true curve is easy to compute exactly. The **trapezoidal rule** approximates the area under f on each small sub-interval by the area under the straight line joining the two endpoints; **Simpson's rule** does better by fitting a parabola through three points at a time, exploiting the curvature information that three points, rather than two, provide.

Formal Definitions

The Root-Finding Problem

Definition. Given a continuous function $f : \mathbb{R} \rightarrow \mathbb{R}$, a **root** is a value r such that $f(r) = 0$. A root r is **simple** if $f'(r) \neq 0$.

Definition (bracket). An interval $[a, b]$ is said to **bracket** a root of f if $f(a)$ and $f(b)$ have opposite signs, i.e. $f(a)f(b) < 0$; by the Intermediate Value Theorem, a continuous f then has at least one root in (a, b) .

Systems of Nonlinear Equations

Definition. For a vector-valued function $\mathbf{F} : \mathbb{R}^p \rightarrow \mathbb{R}^p$, $\mathbf{F}(\mathbf{x}) = (F_1(\mathbf{x}), \dots, F_p(\mathbf{x}))'$, a **root** is a vector $\mathbf{r}$ with $\mathbf{F}(\mathbf{r}) = \mathbf{0}$. The **Jacobian matrix** of $\mathbf{F}$ at $\mathbf{x}$ is

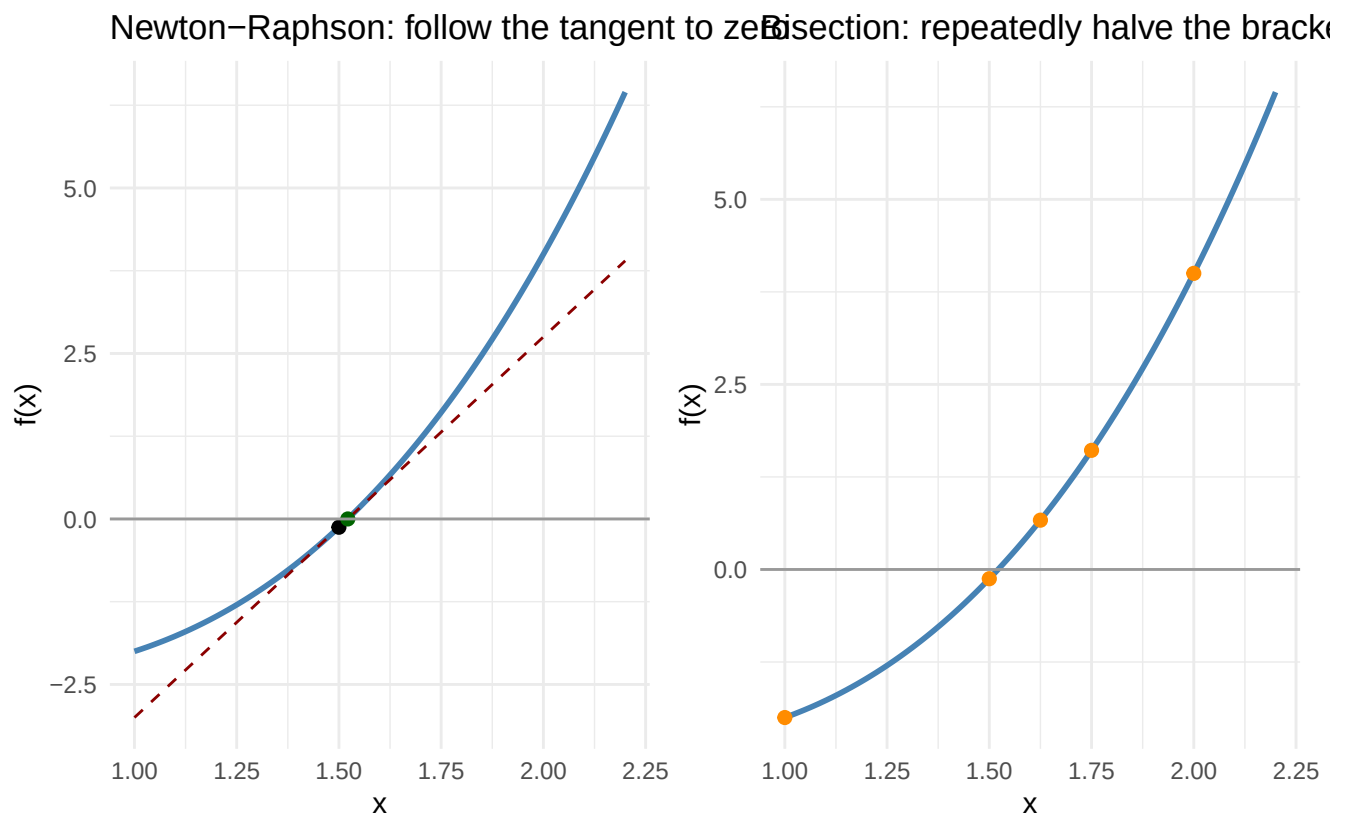


Figure 1: Newton-Raphson (left) jumps to where the tangent line crosses zero, converging much faster near a smooth root than bisection's repeated halving (right).

$$J(\mathbf{x}) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \cdots & \frac{\partial F_1}{\partial x_p} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_p}{\partial x_1} & \cdots & \frac{\partial F_p}{\partial x_p} \end{pmatrix}. \quad (2.1)$$

Numerical Quadrature

Definition. A **quadrature formula** approximates a definite integral by a weighted sum of integrand values at prescribed nodes,

$$\int_a^b f(x) dx \approx \sum_{i=0}^m w_i f(x_i), \quad (2.2)$$

for nodes $a = x_0 < x_1 < \cdots < x_m = b$ and weights w_i determined by the specific rule. A quadrature rule has **degree of precision** d if it integrates every polynomial of degree $\leq d$ exactly, and fails to integrate some polynomial of degree $d + 1$ exactly.

Theorems

Theorem 2.1: Linear Convergence of the Bisection Method

Statement. Let f be continuous on $[a_0, b_0]$ with $f(a_0)f(b_0) < 0$. The bisection method generates a sequence of midpoints c_k satisfying

$$|c_k - r| \leq \frac{b_0 - a_0}{2^{k+1}} \quad (2.3)$$

for some root $r \in (a_0, b_0)$, for every $k = 0, 1, 2, \dots$

Assumptions. f is continuous on $[a_0, b_0]$; only a sign change is required, not differentiability.

Proof. By induction on k , the bracket $[a_k, b_k]$ maintained by the algorithm satisfies $b_k - a_k = (b_0 - a_0)/2^k$: initially this holds for $k = 0$, and at each step the bracket width is exactly halved by construction (the midpoint $c_k = (a_k + b_k)/2$ replaces whichever endpoint does not change sign with it, halving the width). Since $f(a_k)f(b_k) < 0$ throughout (this sign-change property is preserved by construction: the algorithm always keeps the sub-interval on which the sign change persists), the Intermediate Value Theorem guarantees a root $r_k \in (a_k, b_k)$ at every step; because the brackets are nested, all r_k coincide with a single root $r \in (a_0, b_0)$ common to every bracket. Since c_k is the midpoint of $[a_k, b_k]$ and $r \in [a_k, b_k]$,

$$|c_k - r| \leq \frac{b_k - a_k}{2} = \frac{b_0 - a_0}{2^{k+1}},$$

as claimed. ■

Remarks. The bound (2.3) holds unconditionally, with no smoothness requirement on f beyond continuity; this robustness is the bisection method's chief advantage, at the cost of convergence that is always linear (each iteration gains a fixed number of correct bits, not an increasing number).

Examples. A full worked example is given in Section 11.

Applications. Bisection is the standard fallback method inside `uniroot()` in R, used whenever a robust (if slow) root-finder is needed, for example to invert a monotone cumulative distribution function numerically.

Theorem 2.2: Quadratic Convergence of the Newton-Raphson Method

Statement. Let r be a simple root of f ($f(r) = 0$, $f'(r) \neq 0$), with f twice continuously differentiable in a neighbourhood of r . If the Newton-Raphson iterates $x_{n+1} = x_n - f(x_n)/f'(x_n)$ are generated from a starting point x_0 sufficiently close to r , then the errors $e_n = r - x_n$ satisfy

$$e_{n+1} = -\frac{f''(\xi_n)}{2f'(x_n)} e_n^2 \quad (2.4)$$

for some ξ_n between r and x_n , and consequently there exists a constant M (depending on bounds for f'' and f' near r) such that $|e_{n+1}| \leq M e_n^2$, i.e. convergence is (at least) quadratic.

Assumptions. $f'(r) \neq 0$ (simple root); f'' continuous near r ; x_0 close enough to r that all iterates remain in a neighbourhood where $f' \neq 0$.

Proof. By Taylor's theorem with Lagrange remainder, expanding $f(r)$ about x_n ,

$$0 = f(r) = f(x_n) + f'(x_n)(r - x_n) + \frac{f''(\xi_n)}{2}(r - x_n)^2$$

for some ξ_n between x_n and r . Dividing through by $f'(x_n)$ (non-zero for x_n close enough to r , by continuity of f' and $f'(r) \neq 0$),

$$\frac{f(x_n)}{f'(x_n)} + (r - x_n) = -\frac{f''(\xi_n)}{2f'(x_n)}(r - x_n)^2.$$

Since $x_{n+1} = x_n - f(x_n)/f'(x_n)$, the term $f(x_n)/f'(x_n)$ equals $x_n - x_{n+1}$; substituting,

$$(x_n - x_{n+1}) + (r - x_n) = -\frac{f''(\xi_n)}{2f'(x_n)}(r - x_n)^2 \implies r - x_{n+1} = -\frac{f''(\xi_n)}{2f'(x_n)}(r - x_n)^2,$$

which is exactly (2.4). Taking $M = \sup |f''|/\inf(2|f'|)$ over a small neighbourhood of r (finite by continuity of f' , f'' and $f'(r) \neq 0$) gives $|e_{n+1}| \leq M e_n^2$. ■

Remarks. Quadratic convergence means that, once the iterates are close enough to r , the number of correct decimal digits approximately *doubles* at every iteration — a dramatically faster rate than bisection's fixed linear gain, as the numerical example in Section 11 illustrates directly. The requirement that x_0 be “sufficiently close” is essential: Newton-Raphson has no global convergence guarantee analogous to bisection's, and can diverge, cycle, or converge to an unintended root if started poorly or if $f'(x_n)$ comes close to zero during the iteration.

Examples. A full worked example is given in Section 11.

Applications. Newton-Raphson (and its generalization, Fisher scoring, which replaces f' by its expectation) is the standard algorithm used to solve the score equations of maximum likelihood estimation for generalized linear models and countless other parametric statistical models.

Mathematical Derivations

Newton-Raphson for Systems of Nonlinear Equations

The scalar Newton-Raphson update $x_{n+1} = x_n - f(x_n)/f'(x_n)$ generalizes to a system $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ by the same tangent-line (now tangent-hyperplane) reasoning. Expanding $\mathbf{F}$ to first order about $\mathbf{x}_n$,

$$\mathbf{0} \approx \mathbf{F}(\mathbf{x}_n) + J(\mathbf{x}_n)(\mathbf{x}_{n+1} - \mathbf{x}_n),$$

and solving for $\mathbf{x}_{n+1}$ (assuming $J(\mathbf{x}_n)$ is invertible) gives the multivariate update

$$\mathbf{x}_{n+1} = \mathbf{x}_n - J(\mathbf{x}_n)^{-1} \mathbf{F}(\mathbf{x}_n). \quad (2.5)$$

In practice, (2.5) is implemented not by explicitly inverting $J(\mathbf{x}_n)$ (computationally wasteful and numerically less stable) but by solving the linear system $J(\mathbf{x}_n) \boldsymbol{\delta}_n = -\mathbf{F}(\mathbf{x}_n)$ for the step $\boldsymbol{\delta}_n$, and setting $\mathbf{x}_{n+1} = \mathbf{x}_n + \boldsymbol{\delta}_n$. Under the analogous smoothness and non-singularity conditions to the scalar case, the multivariate iteration also converges quadratically near a simple root (a root at which J is non-singular).

Derivation of the Trapezoidal Rule and Its Error

On a single panel $[x_0, x_1]$ of width $h = x_1 - x_0$, the trapezoidal rule replaces f by the straight line P_1 interpolating $(x_0, f(x_0))$ and $(x_1, f(x_1))$, giving

$$\int_{x_0}^{x_1} f(x) dx \approx \int_{x_0}^{x_1} P_1(x) dx = \frac{h}{2} (f(x_0) + f(x_1)). \quad (2.6)$$

The standard linear interpolation error formula gives $f(x) - P_1(x) = \frac{1}{2} f''(\eta(x))(x - x_0)(x - x_1)$ for some $\eta(x)$ between x_0 and x_1 (a consequence of Rolle's theorem applied to the interpolation remainder; see Burden & Faires, 2011, Thm. 3.3). Since $(x - x_0)(x - x_1) \leq 0$ throughout $[x_0, x_1]$, the weighted mean value theorem for integrals gives, for some $\xi \in (x_0, x_1)$,

$$\int_{x_0}^{x_1} [f(x) - P_1(x)] dx = \frac{f''(\xi)}{2} \int_{x_0}^{x_1} (x - x_0)(x - x_1) dx = \frac{f''(\xi)}{2} \left(-\frac{h^3}{6} \right) = -\frac{h^3}{12} f''(\xi), \quad (2.7)$$

using the standard result $\int_{x_0}^{x_1} (x - x_0)(x - x_1) dx = -h^3/6$ (verified directly by substitution $u = x - x_0$: $\int_0^h u(u - h) du = [u^3/3 - hu^2/2]_0^h = h^3/3 - h^3/2 = -h^3/6$). Summing (2.7) over n panels of a composite rule and applying a further mean value argument to the sum of the per-panel errors (each proportional to $h^3 f''(\xi_i)$, summed and recognized as an $O(h)$ -spaced Riemann-type sum) gives the composite trapezoidal error

$$E_{\text{trap}} = -\frac{(b-a)h^2}{12} f''(\xi), \quad \xi \in (a, b), \quad (2.8)$$

confirming that the composite trapezoidal rule has error $O(h^2)$.

Derivation of Simpson's Rule and Its Error

Simpson's rule approximates the integral over a panel $[x_0, x_2]$ of width $2h$, with midpoint $x_1 = x_0 + h$, by fitting a quadratic through the three points. Writing $m = x_1$ and expanding both the exact integral and the quadrature rule in a Taylor series about m makes the derivation self-contained. Using $f(x) = f(m) + f'(m)(x - m) + \frac{1}{2} f''(m)(x - m)^2 + \frac{1}{6} f'''(m)(x - m)^3 + \frac{1}{24} f''''(m)(x - m)^4 + \dots$ and integrating term by term over $[m - h, m + h]$ (odd-power terms vanish by symmetry),

$$\int_{m-h}^{m+h} f(x) dx = 2hf(m) + \frac{h^3}{3} f''(m) + \frac{h^5}{60} f''''(m) + O(h^7). \quad (2.9)$$

For the quadrature side, Simpson's rule is $S = \frac{h}{3} [f(m-h) + 4f(m) + f(m+h)]$; Taylor-expanding $f(m-h)$ and $f(m+h)$ and adding (again the odd terms cancel), $f(m-h) + f(m+h) = 2f(m) + h^2 f''(m) + \frac{h^4}{12} f''''(m) + O(h^6)$, so

$$S = \frac{h}{3} \left[6f(m) + h^2 f''(m) + \frac{h^4}{12} f''''(m) \right] + O(h^7) = 2hf(m) + \frac{h^3}{3} f''(m) + \frac{h^5}{36} f''''(m) + O(h^7). \quad (2.10)$$

Subtracting (2.10) from (2.9), the $f(m)$ and $f''(m)$ terms cancel exactly, leaving

$$E_{\text{Simpson, panel}} = \left(\frac{1}{60} - \frac{1}{36} \right) h^5 f''''(m) + O(h^7) = -\frac{h^5}{90} f''''(\xi) \quad (2.11)$$

for some ξ near m (replacing $f''''(m)$ by $f''''(\xi)$ via the mean value theorem to absorb the $O(h^7)$ remainder). Summing over $n/2$ panels of a composite rule (an even number n of sub-intervals is required) gives the standard composite Simpson error

$$E_{\text{Simpson}} = -\frac{(b-a)h^4}{180} f''''(\xi), \quad \xi \in (a, b), \quad (2.12)$$

showing the composite Simpson's rule has error $O(h^4)$ — two full orders better than the trapezoidal rule's $O(h^2)$ for a comparable number of function evaluations, at the cost of requiring an even number of sub-intervals and one additional order of smoothness (f'''' rather than f'').

Algorithms

Algorithm 2.1: Bisection Method

Purpose. Locate a root of f within a bracket $[a, b]$, with a guaranteed, if slow, rate of convergence.

Input. Function f ; bracket $[a, b]$ with $f(a)f(b) < 0$; tolerance ε .

Output. An approximate root c with $|c - r| < \varepsilon$.

Step-by-step procedure.

1. While $(b - a)/2 > \varepsilon$:
 - (a) compute $c = (a + b)/2$;
 - (b) if $f(a)f(c) < 0$, set $b = c$; otherwise set $a = c$.
2. Return $c = (a + b)/2$.

Pseudo-code.

Algorithm: Bisection Method

Input: f , a , b (with $f(a)f(b) < 0$), epsilon

Output: approximate root c

1. While $(b - a) / 2 > \text{epsilon}$:

$c \leftarrow (a + b) / 2$
 If $f(a) * f(c) < 0$:
 $b \leftarrow c$
 Else:
 $a \leftarrow c$
2. Return $(a + b) / 2$

Flowchart description. Start; check the width of the bracket against the tolerance; if too wide, compute the midpoint, evaluate the function there, and replace whichever endpoint shares its sign with the midpoint; repeat; once the bracket is narrow enough, return the midpoint; end.

Complexity. $O(\log_2((b - a)/\varepsilon))$ iterations, each costing one function evaluation, by Theorem 2.1.

Practical considerations. Bisection requires only a valid initial bracket, and always converges for a continuous function once one is found; it is the natural fallback when Newton-Raphson fails to converge or when derivatives of f are unavailable or unreliable.

Algorithm 2.2: Newton-Raphson Method

Purpose. Locate a root of f rapidly, using derivative information, near a good starting guess.

Input. Function f and its derivative f' ; starting point x_0 ; tolerance ε ; maximum iterations N .

Output. An approximate root x_n .

Step-by-step procedure.

1. For $n = 0, 1, 2, \dots$ until $|x_{n+1} - x_n| < \varepsilon$ or $n = N$: compute $x_{n+1} = x_n - f(x_n)/f'(x_n)$.
2. Return the final iterate.

Pseudo-code.

Algorithm: Newton-Raphson Method

Input: f , f_prime , x_0 , $epsilon$, N

Output: approximate root x

1. $x \leftarrow x_0$
2. For $n = 1$ to N :
 - $x_new \leftarrow x - f(x) / f_prime(x)$
 - If $|x_new - x| < epsilon$:
 - Return x_new
 - $x \leftarrow x_new$
3. Return x (or report non-convergence)

Flowchart description. Start; evaluate f and f' at the current iterate; compute the tangent-line intercept as the new iterate; check for convergence; if not converged and the iteration budget remains, repeat; end.

Complexity. Under quadratic convergence (Theorem 2.2), the number of iterations to reach tolerance ε , once in the basin of quadratic convergence, is $O(\log \log(1/\varepsilon))$ — dramatically fewer than bisection's $O(\log(1/\varepsilon))$ — but each iteration requires evaluating both f and f' .

Practical considerations. Newton-Raphson can fail — diverge, cycle, or converge to the wrong root — if x_0 is poorly chosen or $f'(x_n)$ is close to zero at some iterate; a common safeguard is to combine Newton-Raphson with a bisection fallback (as `uniroot()` and professional root-finders do) to retain global reliability while benefiting from Newton's local speed.

Algorithm 2.3: Newton-Raphson for Systems of Nonlinear Equations

Purpose. Solve $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ for a vector-valued $\mathbf{F}$, generalizing Algorithm 2.2 using (2.5).

Input. Function $\mathbf{F}$ and its Jacobian J ; starting vector $\mathbf{x}_0$; tolerance ε ; maximum iterations N .

Output. An approximate root $\mathbf{x}_n$.

Step-by-step procedure.

1. For $n = 0, 1, 2, \dots$ until $\|\mathbf{x}_{n+1} - \mathbf{x}_n\| < \varepsilon$ or $n = N$: solve the linear system $J(\mathbf{x}_n)\boldsymbol{\delta}_n = -\mathbf{F}(\mathbf{x}_n)$ for $\boldsymbol{\delta}_n$; set $\mathbf{x}_{n+1} = \mathbf{x}_n + \boldsymbol{\delta}_n$.
2. Return the final iterate.

Pseudo-code.

Algorithm: Newton-Raphson for Systems

Input: $\mathbf{F}$, J , $\mathbf{x}_0$, $epsilon$, N

Output: approximate root vector $\mathbf{x}$

```

1. x <- x0
2. For n = 1 to N:
    delta <- solve( J(x), -F(x) )
    x_new <- x + delta
    If norm(x_new - x) < epsilon:
        Return x_new
    x <- x_new
3. Return x (or report non-convergence)

```

Flowchart description. Start; evaluate $\mathbf{F}$ and J at the current vector iterate; solve the resulting linear system for the step; update the iterate; check convergence in norm; repeat if not converged; end.

Complexity. Each iteration costs $O(p^3)$ for solving the $p \times p$ linear system (via Gaussian elimination or LU decomposition), in addition to the cost of evaluating $\mathbf{F}$ and J ; the number of iterations required is again $O(\log \log(1/\varepsilon))$ near a simple root under the multivariate analogue of Theorem 2.2.

Practical considerations. A singular or near-singular Jacobian at any iterate causes the linear solve to fail or become numerically unstable; monitoring the condition number of $J(\mathbf{x}_n)$, or adding a small ridge term when necessary, is standard practice in production implementations (as in the Levenberg-Marquardt modification used for nonlinear least squares).

Algorithm 2.4: Composite Trapezoidal and Simpson's Rules

Purpose. Approximate $\int_a^b f(x) dx$ using evenly spaced function evaluations.

Input. Function f ; interval $[a, b]$; number of sub-intervals n (even, for Simpson's rule).

Output. Approximate integral value.

Step-by-step procedure (Trapezoidal).

1. Set $h = (b - a)/n$ and $x_i = a + ih$, $i = 0, \dots, n$.
2. Compute $T = \frac{h}{2} [f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)]$.

Step-by-step procedure (Simpson's, n even).

1. Set $h = (b - a)/n$ and $x_i = a + ih$, $i = 0, \dots, n$.
2. Compute $S = \frac{h}{3} [f(x_0) + 4 \sum_{i \text{ odd}} f(x_i) + 2 \sum_{i \text{ even}, 0 < i < n} f(x_i) + f(x_n)]$.

Pseudo-code.

Algorithm: Composite Trapezoidal / Simpson's Rule

Input: f, a, b, n

Output: approximate integral

```

1. h <- (b - a) / n
2. x[0..n] <- a + i * h for i = 0..n
3. Trapezoidal:
    T <- h/2 * ( f(x[0]) + 2*sum(f(x[1..n-1])) + f(x[n]) )
4. Simpson (n even):
    S <- h/3 * ( f(x[0]) + 4*sum(f(x[odd i])) + 2*sum(f(x[even i, interior])) + f(x[n]) )
5. Return T (or S)

```

Flowchart description. Start; partition $[a, b]$ into n equal sub-intervals; evaluate f at every node; combine the values with the prescribed weights (uniform for the endpoints and alternating 4-2-4-2 weights for Simpson's interior nodes); return the weighted sum; end.

Complexity. $O(n)$ function evaluations for both rules; Simpson's rule achieves $O(h^4)$ accuracy against the trapezoidal rule's $O(h^2)$ for the same n (hence the same computational cost), making it strictly preferable whenever f'''' is well behaved.

Practical considerations. Both rules degrade for integrands with sharp peaks, discontinuities, or singularities within $[a, b]$; adaptive quadrature (as implemented in R's `integrate()`), which concentrates evaluation points where the integrand varies most rapidly, should be preferred for such integrands over a fixed- n composite rule.

R Programming

```
set.seed(123)

#' Bisection method for root-finding
#'
#' Purpose: implements Algorithm 2.1.
#'
#' Arguments:
#'   target_function  function f for which a root is sought
#'   lower_bound      left endpoint of the bracket
#'   upper_bound      right endpoint of the bracket
#'   tolerance        stopping tolerance on the bracket width
#'
#' Returns:
#'   a list with the approximate root and the iteration history
#'
#' Example:
#'   bisection_method(function(x) x^3 - x - 2, 1, 2, 1e-8)
bisection_method <- function(target_function, lower_bound, upper_bound, tolerance = 1e-8) {
  if (target_function(lower_bound) * target_function(upper_bound) >= 0) {
    stop("The supplied interval does not bracket a root.")
  }

  iteration_history <- numeric(0)
  while ((upper_bound - lower_bound) / 2 > tolerance) {
    midpoint <- (lower_bound + upper_bound) / 2
    iteration_history <- c(iteration_history, midpoint)

    if (target_function(lower_bound) * target_function(midpoint) < 0) {
      upper_bound <- midpoint
    } else {
      lower_bound <- midpoint
    }
  }

  list(root = (lower_bound + upper_bound) / 2, history = iteration_history)
}

#' Newton-Raphson method for root-finding
#'
#' Purpose: implements Algorithm 2.2.
#'
#' Arguments:
```

```

#' target_function      function f
#' target_derivative    function f'
#' starting_point      initial guess x0
#' tolerance            stopping tolerance on successive iterates
#' max_iterations       maximum number of iterations
#'
#' Returns:
#'   a list with the approximate root and the iteration history
#'
#' Example:
#'   newton_raphson_method(
#'     function(x) x^3 - x - 2, function(x) 3 * x^2 - 1, 1.5, 1e-10
#'   )
newton_raphson_method <- function(target_function, target_derivative, starting_point,
                                tolerance = 1e-10, max_iterations = 100) {
  current_point <- starting_point
  iteration_history <- current_point

  for (iteration in seq_len(max_iterations)) {
    next_point <- current_point - target_function(current_point) / target_derivative(current_point)
    iteration_history <- c(iteration_history, next_point)

    if (abs(next_point - current_point) < tolerance) {
      return(list(root = next_point, history = iteration_history))
    }
    current_point <- next_point
  }

  list(root = current_point, history = iteration_history)
}

#' Newton-Raphson method for systems of nonlinear equations
#'
#' Purpose: implements Algorithm 2.3.
#'
#' Arguments:
#'   system_function      function returning the vector F(x)
#'   jacobian_function    function returning the Jacobian matrix J(x)
#'   starting_vector      initial guess x0
#'   tolerance            stopping tolerance on the step norm
#'   max_iterations       maximum number of iterations
#'
#' Returns:
#'   a list with the approximate root vector and the iteration history
#'
#' Example:
#'   newton_raphson_system(
#'     function(v) c(v[1]^2 + v[2]^2 - 4, v[1] * v[2] - 1),
#'     function(v) matrix(c(2 * v[1], v[2], 2 * v[2], v[1]), nrow = 2),
#'     c(1.8, 0.6)
#'   )
newton_raphson_system <- function(system_function, jacobian_function, starting_vector,
                                tolerance = 1e-10, max_iterations = 100) {

```

```

current_vector <- starting_vector
iteration_history <- list(current_vector)

for (iteration in seq_len(max_iterations)) {
  step_vector <- solve(jacobian_function(current_vector), -system_function(current_vector))
  next_vector <- current_vector + step_vector
  iteration_history[[iteration + 1]] <- next_vector

  if (sqrt(sum((next_vector - current_vector)^2)) < tolerance) {
    return(list(root = next_vector, history = iteration_history))
  }
  current_vector <- next_vector
}

list(root = current_vector, history = iteration_history)
}

#' Composite trapezoidal rule
#'
#' Purpose: implements the trapezoidal part of Algorithm 2.4.
#'
#' Arguments:
#'   integrand_function  function to integrate
#'   lower_limit          lower limit of integration
#'   upper_limit          upper limit of integration
#'   number_of_intervals number of sub-intervals
#'
#' Returns:
#'   the approximate integral (numeric scalar)
#'
#' Example:
#'   trapezoidal_rule(function(x) x^2, 0, 2, 4)
trapezoidal_rule <- function(integrand_function, lower_limit, upper_limit, number_of_intervals) {
  step_size <- (upper_limit - lower_limit) / number_of_intervals
  grid_points <- lower_limit + (0:number_of_intervals) * step_size
  function_values <- integrand_function(grid_points)

  step_size / 2 * (function_values[1] + 2 * sum(function_values[2:number_of_intervals]) +
    function_values[number_of_intervals + 1])
}

#' Composite Simpson's rule
#'
#' Purpose: implements the Simpson's part of Algorithm 2.4.
#'
#' Arguments:
#'   integrand_function  function to integrate
#'   lower_limit          lower limit of integration
#'   upper_limit          upper limit of integration
#'   number_of_intervals number of sub-intervals (must be even)
#'
#' Returns:
#'   the approximate integral (numeric scalar)

```

```

#'
#' Example:
#' simpsons_rule(function(x) x^2, 0, 2, 4)
simpsons_rule <- function(integrand_function, lower_limit, upper_limit, number_of_intervals) {
  if (number_of_intervals %% 2 != 0) {
    stop("Simpson's rule requires an even number of sub-intervals.")
  }

  step_size <- (upper_limit - lower_limit) / number_of_intervals
  grid_points <- lower_limit + (0:number_of_intervals) * step_size
  function_values <- integrand_function(grid_points)

  odd_indices <- seq(2, number_of_intervals, by = 2)
  even_indices <- if (number_of_intervals >= 4) {
    seq(3, number_of_intervals - 1, by = 2)
  } else {
    integer(0)
  }

  step_size / 3 * (
    function_values[1] +
    4 * sum(function_values[odd_indices]) +
    2 * sum(function_values[even_indices]) +
    function_values[number_of_intervals + 1]
  )
}

```

Simulation Studies

Empirical Convergence Rates: Bisection vs. Newton-Raphson

```

set.seed(123)

target_f <- function(x) x^3 - x - 2
target_f_prime <- function(x) 3 * x^2 - 1
true_root <- uniroot(target_f, c(1, 2), tol = 1e-14)$root

bisection_result <- bisection_method(target_f, 1, 2, tolerance = 1e-12)
newton_result <- newton_raphson_method(target_f, target_f_prime, 1.5, tolerance = 1e-14)

bisection_errors <- abs(bisection_result$history - true_root)
newton_errors <- abs(newton_result$history - true_root)

tibble(
  Iteration = seq_along(bisection_errors),
  Bisection_Error = bisection_errors
) %>%
  slice(1:8) %>%
  kable(digits = 8, caption = "Bisection error by iteration (halves each step)")

```

Table 1: Bisection error by iteration (halves each step)

Iteration	Bisection_Error
1	0.02137971
2	0.22862029
3	0.10362029
4	0.04112029
5	0.00987029
6	0.00575471
7	0.00205779
8	0.00184846

```
tibble(
  Iteration = seq_along(newton_errors) - 1,
  Newton_Error = newton_errors
) %>%
  kable(digits = 12, caption = "Newton-Raphson error by iteration (squares each step, once close)")
```

Table 2: Newton-Raphson error by iteration (squares each step, once close)

Iteration	Newton_Error
0	2.137971e-02
1	3.594236e-04
2	9.916000e-08
3	0.000000e+00
4	0.000000e+00

Convergence Order of Trapezoidal and Simpson's Rules

```
set.seed(123)

test_integrand <- function(x) exp(-x^2)
true_value <- integrate(test_integrand, 0, 1)$value

interval_counts <- c(2, 4, 8, 16, 32, 64, 128)
trap_errors <- vapply(
  interval_counts, function(n) abs(trapezoidal_rule(test_integrand, 0, 1, n) - true_value), numeric(1)
)
simpson_errors <- vapply(
  interval_counts, function(n) abs(simpsons_rule(test_integrand, 0, 1, n) - true_value), numeric(1)
)

tibble(
  n = interval_counts, Trapezoidal_Error = trap_errors, Simpson_Error = simpson_errors
) %>%
  kable(digits = 10, caption = "Integration error for the integral of exp(-x^2) on [0,1] (true value 0.)")
```

Table 3: Integration error for the integral of $\exp(-x^2)$ on $[0,1]$ (true value 0.7468241)

n	Trapezoidal_Error	Simpson_Error
2	0.0154538810	0.0003562961
4	0.0038400350	0.0000312470
8	0.0009585180	0.0000019877
16	0.0002395360	0.0000001246
32	0.0000598782	0.0000000078
64	0.0000149692	0.0000000005
128	0.0000037423	0.0000000000

Graphs

```
convergence_df <- tibble(
  n = rep(interval_counts, 2),
  error = c(trap_errors, simpson_errors),
  method = rep(c("Trapezoidal", "Simpson"), each = length(interval_counts))
)
```

```
ggplot(convergence_df, aes(x = n, y = error, colour = method)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    title = "Quadrature error vs. number of sub-intervals (log-log scale)",
    x = "Number of sub-intervals (n)", y = "Absolute error"
  ) +
  theme_minimal(base_size = 11)
```

```
x_grid_plot <- seq(1, 2, length.out = 200)
```

```
ggplot() +
  geom_line(data = tibble(x = x_grid_plot, y = target_f(x_grid_plot)), aes(x, y), colour = "steelblue")
  geom_hline(yintercept = 0, colour = "grey60") +
  geom_point(
    data = tibble(x = newton_result$history, y = target_f(newton_result$history)),
    aes(x, y), colour = "darkred", size = 2
  ) +
  geom_path(
    data = tibble(x = newton_result$history, y = target_f(newton_result$history)),
    aes(x, y), colour = "darkred", linetype = "dotted"
  ) +
  labs(title = "Newton-Raphson iterates on  $f(x) = x^3 - x - 2$ ", x = "x", y = "f(x)") +
  theme_minimal(base_size = 11)
```

Numerical Examples

Example 1: Bisection method.

Problem. Find a root of $f(x) = x^3 - x - 2$ in $[1, 2]$, performing three iterations of bisection by hand.

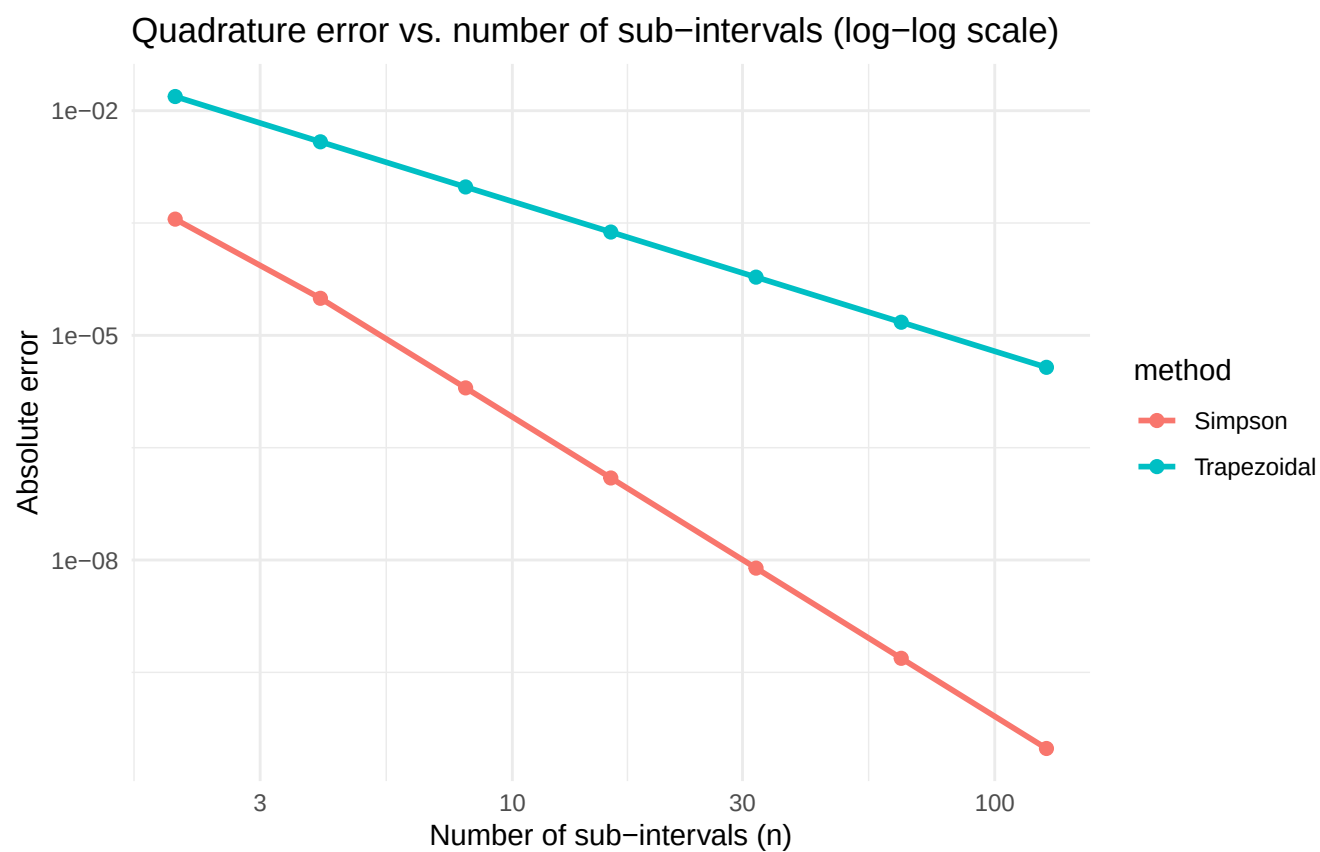


Figure 2: Log-log plot of quadrature error against number of sub-intervals: the trapezoidal rule's slope of -2 and Simpson's rule's slope of -4 confirm the $O(h^2)$ and $O(h^4)$ rates of Section 8.

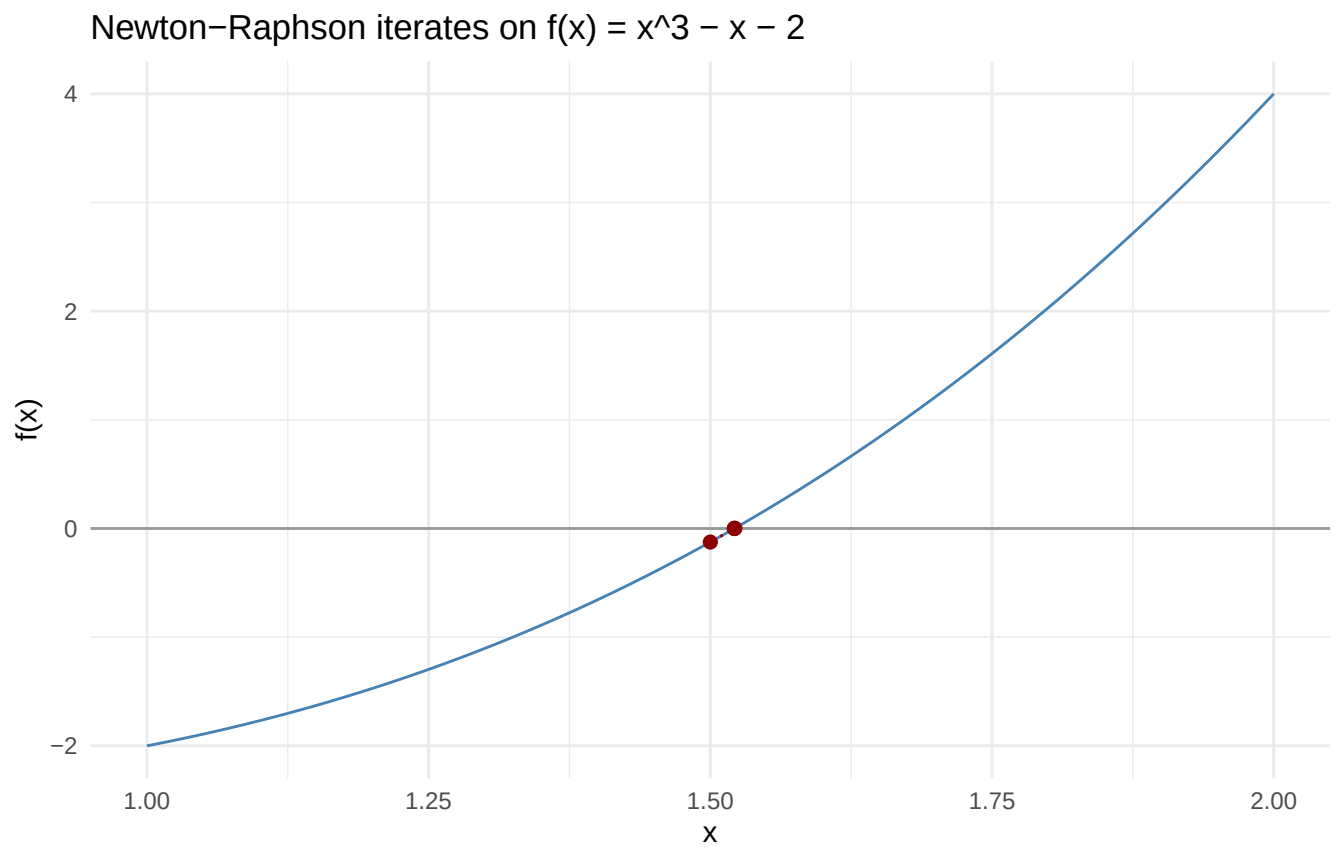


Figure 3: Newton-Raphson iterates converge quadratically to the true root, visualized against the target function.

Solution. $f(1) = -2$, $f(2) = 4$ (opposite signs, bracket valid).

$$\begin{aligned} c_1 &= 1.5, & f(1.5) &= -0.125 \text{ (same sign as } f(1)) \Rightarrow \text{new bracket } [1.5, 2] \\ c_2 &= 1.75, & f(1.75) &= 1.609375 \text{ (opposite sign)} \Rightarrow \text{new bracket } [1.5, 1.75] \\ c_3 &= 1.625, & f(1.625) &= 0.666016 \text{ (opposite sign)} \Rightarrow \text{new bracket } [1.5, 1.625] \end{aligned}$$

Interpretation. After three iterations the bracket has shrunk from width 1 to width $0.125 = 1/8$, exactly matching the halving predicted by Theorem 2.1; the true root, $r \approx 1.521380$, indeed lies in $[1.5, 1.625]$.

Example 2: Newton-Raphson method.

Problem. Using the same f , with $f'(x) = 3x^2 - 1$, perform two iterations of Newton-Raphson from $x_0 = 1.5$.

Solution.

$$\begin{aligned} x_1 &= 1.5 - \frac{f(1.5)}{f'(1.5)} = 1.5 - \frac{-0.125}{5.75} = 1.521739, \\ x_2 &= x_1 - \frac{f(x_1)}{f'(x_1)} = 1.521739 - \frac{0.002137}{5.94707} = 1.521380. \end{aligned}$$

Interpretation. Two Newton-Raphson iterations already match the true root $r = 1.521380$ to six decimal places, whereas three bisection iterations (Example 1) had only localized the root to the interval $[1.5, 1.625]$ — a striking illustration of the practical difference between linear and quadratic convergence.

Example 3: Newton-Raphson for a system.

Problem. Solve $x^2 + y^2 = 4$, $xy = 1$ starting from $(x_0, y_0) = (1.8, 0.6)$, performing one iteration by hand.

Solution. $\mathbf{F}(1.8, 0.6) = (1.8^2 + 0.6^2 - 4, 1.8 \times 0.6 - 1) = (-0.4, 0.08)$. The Jacobian is $J = \begin{pmatrix} 2x & 2y \\ y & x \end{pmatrix}$, so $J(1.8, 0.6) = \begin{pmatrix} 3.6 & 1.2 \\ 0.6 & 1.8 \end{pmatrix}$, with determinant $3.6(1.8) - 1.2(0.6) = 5.76$. Solving $J\delta = -\mathbf{F}$,

$$\delta = \frac{1}{5.76} \begin{pmatrix} 1.8 & -1.2 \\ -0.6 & 3.6 \end{pmatrix} \begin{pmatrix} 0.4 \\ -0.08 \end{pmatrix} = \frac{1}{5.76} \begin{pmatrix} 0.816 \\ -0.528 \end{pmatrix} = \begin{pmatrix} 0.141667 \\ -0.091667 \end{pmatrix},$$

so $(x_1, y_1) = (1.8 + 0.141667, 0.6 - 0.091667) = (1.941667, 0.508333)$.

Interpretation. Checking, $x_1^2 + y_1^2 = 4.0285$ and $x_1 y_1 = 0.9870$, both markedly closer to the target values 4 and 1 than the starting point; further iterations converge quadratically to the true solution $(\sqrt{2 + \sqrt{3}}, 1/\sqrt{2 + \sqrt{3}}) \approx (1.931852, 0.517638)$.

Example 4: Trapezoidal and Simpson's rule.

Problem. Approximate $\int_0^2 x^2 dx$ (true value $8/3 = 2.666667$) using $n = 4$ sub-intervals ($h = 0.5$) with both rules.

Solution. Nodes: 0, 0.5, 1, 1.5, 2 with values 0, 0.25, 1, 2.25, 4.

$$T = \frac{0.5}{2} [0 + 2(0.25 + 1 + 2.25) + 4] = 0.25(11) = 2.75.$$

$$S = \frac{0.5}{3} [0 + 4(0.25) + 2(1) + 4(2.25) + 4] = \frac{0.5}{3}(16) = 2.666667.$$

Interpretation. Simpson's rule reproduces the true value *exactly*, because $f(x) = x^2$ is a polynomial of degree $2 < 4$ and Simpson's rule has degree of precision 3 (it integrates every cubic exactly, hence certainly every

quadratic); the trapezoidal rule, exact only for linear integrands, overestimates the true value by 0.0833, consistent with the error formula (2.8), since $f''(x) = 2 > 0$ makes the trapezoidal rule systematically overestimate a convex function.

Applications

- **Statistics.** Newton-Raphson (or its close relative, Fisher scoring) solves the score equations of maximum likelihood estimation for logistic regression, Poisson regression, and virtually every generalized linear model fit by standard software.
- **Machine Learning.** Newton's method and its quasi-Newton variants (BFGS, L-BFGS) are core second-order optimization algorithms; the Newton-Raphson system update (2.5) is precisely the Newton step used in optimizing a smooth loss function, applied to its gradient.
- **Data Science.** Numerical integration is used to compute marginal densities, expectations, and normalizing constants of fitted probability models that lack closed-form integrals.
- **Finance.** Computing implied volatility from an option's market price is a classical one-dimensional root-finding problem, almost universally solved by Newton-Raphson or a safeguarded bisection hybrid.
- **Reliability Engineering.** Solving for a system's failure time or reliability threshold from a nonlinear hazard or reliability function often requires root-finding when no closed-form inverse exists.
- **Monte Carlo Methods.** Numerical quadrature provides the deterministic benchmark against which the Monte Carlo integration methods of Unit 4.3 are compared for low-dimensional integrals.
- **Bayesian Statistics.** The Laplace approximation to a posterior density requires locating the posterior mode via Newton-Raphson (or a quasi-Newton method) and evaluating the Hessian there.
- **Operations Research.** Root-finding is used to solve for equilibrium points or break-even conditions in nonlinear resource allocation and inventory models.
- **Engineering.** Newton-Raphson for systems is the standard method for solving the nonlinear equilibrium equations arising in structural and circuit analysis.

Advantages

- Bisection is completely robust: given a valid bracket, it is mathematically guaranteed to converge, requiring only continuity of f .
- Newton-Raphson converges very rapidly (quadratically) once near a simple root, needing very few iterations for high precision.
- Simpson's rule achieves a substantially higher order of accuracy than the trapezoidal rule at essentially the same computational cost per additional node.

Limitations

- Bisection requires an initial bracket with a sign change, which may not always be easy to locate, particularly for functions with multiple close roots or no sign change at a simple root (a root of even multiplicity).
- Newton-Raphson requires the derivative f' (or the Jacobian, for systems), can diverge or converge to an unintended root from a poor starting point, and can fail outright if $f'(x_n)$ is ever zero or numerically close to it during the iteration.
- Fixed- n quadrature rules such as trapezoidal and Simpson's rules degrade sharply for integrands with singularities, sharp peaks, or discontinuities, and provide no built-in error estimate without additional (e.g. Richardson extrapolation) machinery.

Practical Tips

- **Prefer a hybrid strategy in production code.** Combining Newton-Raphson's speed with bisection's robustness (falling back to a bisection step whenever a Newton step would leave the current bracket) is

standard practice, as implemented internally by professional root-finders including R's `uniroot()`.

- **Always verify convergence, not just iteration count.** Checking both $|x_{n+1} - x_n|$ and $|f(x_{n+1})|$ against separate tolerances guards against premature termination when the two quantities behave differently (as can happen near a root with a very flat or very steep derivative).
- **Use Simpson's rule by default over the trapezoidal rule** for smooth integrands, since it is essentially free — the same number of function evaluations yields two additional orders of accuracy.
- **Use `integrate()`'s adaptive quadrature, not a fixed- n composite rule, for integrands with peaks or singularities.** Fixed-node rules waste evaluations on flat regions and under-resolve rapidly varying regions.
- **When implementing Newton-Raphson for systems, monitor the condition number of the Jacobian.** A poorly conditioned or singular Jacobian at any iterate is a common, silent cause of numerical failure that a simple convergence check may not reveal until many wasted iterations have passed.

Summary

- The bisection method locates a root by repeatedly halving a bracket known to contain a sign change; Theorem 2.1 establishes its unconditional linear convergence.
- The Newton-Raphson method follows the tangent line to a root and, near a simple root, converges quadratically (Theorem 2.2), roughly doubling the number of correct digits per iteration.
- Newton-Raphson generalizes to systems of nonlinear equations via the Jacobian matrix, replacing division by f' with solving a linear system involving J .
- The trapezoidal and Simpson's rules approximate a definite integral using linear and quadratic local interpolation respectively, with error terms $O(h^2)$ and $O(h^4)$ derived explicitly in Section 8, and verified empirically by the log-log convergence plot of Section 10.
- These deterministic numerical methods underlie the practical computation of maximum likelihood estimates and intractable integrals throughout statistical computing.

Key Formulae

- Bisection error bound: $|c_k - r| \leq (b_0 - a_0)/2^{k+1}$.
- Newton-Raphson update: $x_{n+1} = x_n - f(x_n)/f'(x_n)$; quadratic error recursion $e_{n+1} = -\frac{f''(\xi_n)}{2f'(x_n)}e_n^2$.
- Newton-Raphson for systems: $\mathbf{x}_{n+1} = \mathbf{x}_n - J(\mathbf{x}_n)^{-1}\mathbf{F}(\mathbf{x}_n)$.
- Trapezoidal rule: $T = \frac{h}{2}[f(x_0) + 2\sum f(x_i) + f(x_n)]$, error $-\frac{(b-a)h^2}{12}f''(\xi)$.
- Simpson's rule: $S = \frac{h}{3}[f(x_0) + 4\sum_{\text{odd}} f(x_i) + 2\sum_{\text{even}} f(x_i) + f(x_n)]$, error $-\frac{(b-a)h^4}{180}f''''(\xi)$.

Key Terms

- **Bisection method.** A bracketing root-finding method that repeatedly halves an interval known to contain a sign change.
- **Bracket.** An interval on which a continuous function changes sign, guaranteeing a root within it.
- **Degree of precision.** The highest polynomial degree a quadrature rule integrates exactly.
- **Jacobian matrix.** The matrix of first partial derivatives of a vector-valued function.
- **Newton-Raphson method.** A root-finding method that follows the tangent line (or hyperplane) at the current iterate to a new estimate.
- **Quadratic convergence.** A convergence rate in which the error at each step is proportional to the square of the previous error.
- **Simpson's rule.** A quadrature formula based on local quadratic (parabolic) interpolation.
- **Trapezoidal rule.** A quadrature formula based on local linear interpolation.

Exercises

Multiple Choice Questions

1. The bisection method requires
 - (a) a derivative of f
 - (b) a bracket $[a, b]$ with $f(a)f(b) < 0$
 - (c) a Hessian matrix
 - (d) a Taylor series expansion
2. The bisection method's convergence rate is
 - (a) quadratic
 - (b) cubic
 - (c) linear
 - (d) exponential
3. The Newton-Raphson update formula is
 - (a) $x_{n+1} = x_n + f(x_n)/f'(x_n)$
 - (b) $x_{n+1} = x_n - f(x_n)/f'(x_n)$
 - (c) $x_{n+1} = x_n - f'(x_n)/f(x_n)$
 - (d) $x_{n+1} = x_n - f(x_n)f'(x_n)$
4. Newton-Raphson achieves quadratic convergence provided
 - (a) f is continuous only
 - (b) the root is simple and f'' is continuous nearby
 - (c) f is a polynomial of even degree
 - (d) the starting point is exactly the root
5. The Newton-Raphson update for a system of equations replaces division by f' with
 - (a) multiplication by J
 - (b) solving a linear system involving the Jacobian J
 - (c) computing eigenvalues of J
 - (d) transposing J
6. The trapezoidal rule's composite error is of order
 - (a) $O(h)$
 - (b) $O(h^2)$
 - (c) $O(h^3)$
 - (d) $O(h^4)$
7. Simpson's rule's composite error is of order
 - (a) $O(h^2)$
 - (b) $O(h^3)$
 - (c) $O(h^4)$
 - (d) $O(h^5)$
8. Simpson's rule requires
 - (a) an odd number of sub-intervals
 - (b) an even number of sub-intervals
 - (c) exactly 2 sub-intervals
 - (d) no restriction on the number of sub-intervals
9. Simpson's rule integrates every polynomial of degree up to
 - (a) 1
 - (b) 2
 - (c) 3
 - (d) 4
10. A common safeguard against Newton-Raphson divergence is to
 - (a) always start at $x_0 = 0$
 - (b) combine it with a bisection fallback
 - (c) use only the trapezoidal rule instead
 - (d) increase the tolerance to a very large value

Answers. 1(b), 2(c), 3(b), 4(b), 5(b), 6(b), 7(c), 8(b), 9(c), 10(b).

Short Answer Questions

1. State the bracketing condition required to start the bisection method, and explain why it guarantees a root exists.
2. Explain, in one or two sentences, the geometric idea behind the Newton-Raphson method.
3. Write the multivariate Newton-Raphson update formula and identify the role of the Jacobian matrix.
4. State the error order of the trapezoidal rule and of Simpson's rule, and explain which is more accurate for the same n .
5. Explain why Simpson's rule requires an even number of sub-intervals.
6. Why can Newton-Raphson fail even when a root exists nearby?
7. Explain the difference between linear and quadratic convergence in terms of the number of correct digits gained per iteration.
8. Why is bisection often used as a fallback for Newton-Raphson in production software?
9. Give one statistical application of Newton-Raphson distinct from those emphasized in this unit.
10. Explain why Simpson's rule integrates x^2 exactly, as verified in Example 4.

Long Answer Questions

1. State and prove Theorem 2.1 (linear convergence of bisection).
2. State and prove Theorem 2.2 (quadratic convergence of Newton-Raphson), explaining the role of Taylor's theorem in the proof.
3. Derive the composite trapezoidal rule error formula, showing every step from the linear interpolation error to the final composite bound.
4. Derive the single-panel Simpson's rule error formula using the Taylor expansion approach of Section 8.2, showing every algebraic step.
5. Compare bisection and Newton-Raphson in terms of robustness, speed, and information required about f , and describe a scenario in which each would be preferred.
6. Derive the Newton-Raphson update formula for a system of two nonlinear equations from the first-order Taylor expansion of $\mathbf{F}$, explaining each step.
7. Discuss the role of Newton-Raphson (or Fisher scoring) in maximum likelihood estimation, explaining how the score equation and the observed or expected information relate to f and f' in the scalar case.
8. Explain why fixed- n trapezoidal and Simpson's rules can perform poorly for integrands with sharp peaks or singularities, and describe how adaptive quadrature addresses this limitation.
9. Using the numerical example of Section 11 (Example 4), explain why the trapezoidal rule systematically overestimates the integral of a convex function, relating your answer to the sign of the error term (2.8).
10. Discuss the practical safeguards used in professional root-finding software to combine the reliability of bisection with the speed of Newton-Raphson.

Programming Exercises

1. Implement `bisection_method()` and `newton_raphson_method()` from first principles (already provided) and reproduce the manual iterations of Examples 1 and 2 exactly.
2. Implement `newton_raphson_system()` from first principles (already provided) and verify the manual first iteration of Example 3, then run the algorithm to convergence and confirm it reaches the true solution given in the interpretation of that example.
3. Implement `trapezoidal_rule()` and `simpsons_rule()` from first principles (already provided) and reproduce the manual calculation of Example 4 exactly.
4. Reproduce the log-log convergence plot of Section 10 for a different smooth integrand of your choosing, and verify the fitted slopes are close to -2 (trapezoidal) and -4 (Simpson).
5. Use Newton-Raphson to compute the maximum likelihood estimate of the rate parameter of an Exponential distribution from simulated data, and verify your answer against the known closed-form MLE $1/\bar{x}$.

6. Use `newton_raphson_system()` to fit, by maximum likelihood, the two parameters of a simple two-parameter distribution of your choosing (for example, a Gamma distribution) to simulated data, comparing your result to R's built-in `optim()` or `fitdistr()`.
7. Write a function that applies Newton-Raphson to invert the standard Normal cumulative distribution function (i.e. compute $\Phi^{-1}(u)$ without using `qnorm()`), and compare its output and speed to `qnorm()` for a range of u values.
8. Investigate Newton-Raphson divergence empirically: for $f(x) = x^3 - 2x + 2$, show that starting from $x_0 = 0$ causes the iteration to cycle indefinitely, while starting from $x_0 = -2$ converges; explain the difference using a plot of f and its tangent lines.
9. Implement Richardson extrapolation for the trapezoidal rule (combining the estimates at n and $2n$ sub-intervals to cancel the leading error term) and verify empirically that it achieves an error order comparable to Simpson's rule.
10. Write a function comparing the number of function evaluations required by bisection, Newton-Raphson, and R's built-in `uniroot()` to reach a fixed tolerance for a common test function, and tabulate the results.

Mini Projects

1. **A robust hybrid root-finder.** Implement a root-finding function that combines bisection and Newton-Raphson: at each step, attempt a Newton-Raphson update, but fall back to a bisection step whenever the Newton step would leave the current bracket or the derivative is too close to zero. Test your hybrid method against plain Newton-Raphson on at least three test functions chosen to make plain Newton-Raphson struggle (for example, functions with nearby inflection points or very flat derivatives near the root), and report the iteration counts and reliability of each approach.
2. **Numerical integration for a Bayesian normalizing constant.** Consider a posterior density known only up to a normalizing constant, $\pi(\theta) \propto L(\theta)p(\theta)$, for a likelihood L and prior p of your choosing (for a single parameter θ). Use the composite Simpson's rule to compute the normalizing constant $\int \pi(\theta) d\theta$ over a suitable range, verify convergence of your answer as n increases, and use the normalized posterior to compute the posterior mean via a second application of Simpson's rule. Compare your results to a Monte Carlo integration estimate (a preview of Unit 4.3).

Difficulty labels. Multiple choice, short answer: **Easy**. Long answer questions 3, 4, 6, 8, 9, programming exercises 1-6: **Medium**. Long answer questions 1, 2, 5, 7, 10, programming exercises 7-10: **Medium to Advanced**. Mini projects: **Advanced**.

References

- Burden, R. L., & Faires, J. D. (2011). *Numerical analysis* (9th ed.). Brooks/Cole.
- Kelley, C. T. (2003). *Solving nonlinear equations with Newton's method*. SIAM.
- Nocedal, J., & Wright, S. J. (2006). *Numerical optimization* (2nd ed.). Springer.
- Rizzo, M. L. (2019). *Statistical computing with R* (2nd ed.). CRC Press.
- Ross, S. M. (2020). *Simulation* (6th ed.). Academic Press.